
Ottimizzazione su Reti

Capitolo 9

Laboratorio Python e NetworkX

Corso di Laurea Magistrale in Ingegneria Gestionale
Sapienza Università di Roma

Giampaolo Liuzzi
Dipartimento di Ingegneria Informatica, Automatica e Gestionale

Indice

1	Obiettivi del laboratorio	1
2	Ambiente di lavoro	1
3	Convenzioni comuni per i dati	1
3.1	Liste di archi e tabelle	2
4	Validazione dell'istanza	2
5	Verifica indipendente delle soluzioni	3
5.1	Bilanci e capacità	3
5.2	Certificati specifici	3
6	Mapa delle principali funzioni NetworkX	4
7	Separare algoritmo e libreria	4
8	Visualizzazione delle reti	4
9	Test automatici	5
10	Struttura della repository GitHub	5
11	Riproducibilità e stile	6
12	Errori frequenti	6
13	Esercizi di laboratorio	7
14	Sintesi del capitolo	7

Obiettivi di apprendimento

Al termine del capitolo lo studente dovrebbe essere in grado di:

- rappresentare in modo uniforme grafi e istanze di ottimizzazione;
- separare dati, algoritmo, verifica e visualizzazione;
- controllare bilanci, capacità, costi e certificati di ottimalità;
- usare correttamente le principali funzioni NetworkX;
- realizzare visualizzazioni leggibili di flussi, cammini, alberi e tagli;
- scrivere test automatici per piccole istanze;
- organizzare notebook e codice in una repository riproducibile.

1 Obiettivi del laboratorio

Il laboratorio computazionale non serve soltanto a ottenere un numero. Una buona soluzione deve rendere espliciti:

1. il modello matematico rappresentato dal grafo;
2. le convenzioni usate per nodi, archi e attributi;
3. l'algoritmo o la funzione di libreria impiegata;
4. i controlli indipendenti sul risultato;
5. un certificato di ottimalità quando disponibile.

NetworkX offre strutture dati e algoritmi affidabili, ma non verifica se il grafo costruito descrive correttamente il problema reale. Modellazione e validazione restano responsabilità dell'utente.

2 Ambiente di lavoro

Per gli esempi sono sufficienti Python, NetworkX e Matplotlib; NumPy e pandas sono utili per matrici e tabelle.

Listing 1: Importazioni comuni

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

È opportuno registrare le versioni nel file delle dipendenze e usare un ambiente virtuale. I notebook devono poter essere eseguiti dall'inizio alla fine senza operazioni manuali nascoste.

Python/NetworkX

`nx.Graph` rappresenta grafi non orientati, `nx.DiGraph` grafi orientati. `MultiGraph` e `MultiDiGraph` ammettono archi paralleli, ma non tutte le funzioni di ottimizzazione li accettano.

3 Convenzioni comuni per i dati

Adottiamo nomi di attributi coerenti con NetworkX:

Oggetto	Attributo	Significato
nodo	demand	b_i : positivo domanda, negativo offerta
arco	weight	costo, lunghezza o durata
arco	capacity	capacità superiore
arco	lower	limite inferiore, gestito esplicitamente
arco	flow	flusso candidato o calcolato

Convenzioni del progetto

La matrice di incidenza ha -1 all'origine e $+1$ alla destinazione. Pertanto

$$Ax = b, \quad b_i > 0 \text{ domanda}, \quad b_i < 0 \text{ offerta}.$$

L'attributo **demand** coincide direttamente con b_i : non deve essere cambiato di segno.

3.1 Liste di archi e tabelle

Una piccola istanza può essere definita con tuple; per dati esterni è preferibile una tabella CSV con colonne stabili.

Listing 2: Costruzione da una tabella di archi

```
edges = pd.DataFrame([
    {"tail": 1, "head": 2, "cost": 4, "capacity": 10},
    {"tail": 1, "head": 3, "cost": 2, "capacity": 7},
    {"tail": 2, "head": 3, "cost": 1, "capacity": 5},
])

G = nx.DiGraph()
for row in edges.itertuples(index=False):
    G.add_edge(row.tail, row.head,
               weight=row.cost, capacity=row.capacity)
```

4 Validazione dell'istanza

I controlli devono precedere la chiamata all'algoritmo. Verifichiamo almeno:

- tipo di grafo e presenza di nodi e archi;
- attributi obbligatori e valori numerici finiti;
- capacità non negative e limiti inferiori non superiori alle capacità;
- somma delle domande uguale a zero per il flusso a costo minimo;
- esistenza dei nodi sorgente e destinazione;
- aciclicità per algoritmi su DAG e CPM;
- non negatività dei pesi prima di usare Dijkstra.

Listing 3: Validatore essenziale per una rete di flusso

```
def validate_flow_instance(G):
    if not G.is_directed():
        raise ValueError("La rete deve essere orientata")
```

```

total_demand = sum(G.nodes[v].get("demand", 0) for v in G)
if total_demand != 0:
    raise ValueError("La somma delle domande deve essere zero")

for i, j, data in G.edges(data=True):
    cap = data.get("capacity", float("inf"))
    low = data.get("lower", 0)
    if low < 0 or cap < low:
        raise ValueError(f"Limiti non validi su {(i, j)}")

```

5 Verifica indipendente delle soluzioni

5.1 Bilanci e capacità

Un flusso candidato va controllato senza affidarsi al fatto che la funzione sia terminata correttamente.

Listing 4: Verifica di un flusso nella convenzione $Ax = b$

```

def check_flow(G, flow, tol=1e-9):
    balance = {v: 0.0 for v in G}
    cost = 0.0

    for i, j, data in G.edges(data=True):
        x = flow[i][j]
        low = data.get("lower", 0.0)
        cap = data.get("capacity", float("inf"))
        if x < low - tol or x > cap + tol:
            return False, None
        balance[i] -= x
        balance[j] += x
        cost += data.get("weight", 0.0) * x

    ok = all(abs(balance[v] - G.nodes[v].get("demand", 0.0))
             <= tol for v in G)
    return ok, cost

```

5.2 Certificati specifici

Problema	Controllo o certificato
flusso a costo minimo	bilanci, limiti, costo e costi ridotti
massimo flusso	valore del flusso uguale alla capacità del taglio
cammino minimo	costo del cammino e disuguaglianze di Bellman
albero	connessione, aciclicità e $n - 1$ spigoli
CPM	ricorrenze avanti/indietro e slittamenti non negativi

6 Mappa delle principali funzioni NetworkX

Problema	Funzione	Attributo
componenti connesse	<code>connected_components</code>	–
ordinamento topologico	<code>topological_sort</code>	–
albero ricoprente minimo	<code>minimum_spanning_tree</code>	<code>weight</code>
flusso a costo minimo	<code>network_simplex</code>	<code>demand</code> , <code>weight</code> , <code>capacity</code>
massimo flusso	<code>maximum_flow</code>	<code>capacity</code>
taglio minimo	<code>minimum_cut</code>	<code>capacity</code>
cammino minimo	<code>shortest_path</code>	<code>weight</code>
Dijkstra	<code>single_source_dijkstra</code>	<code>weight</code>
Bellman-Ford	<code>single_source_bellman_ford</code>	<code>weight</code>
cammino massimo in DAG	<code>dag_longest_path</code>	<code>weight</code>

Attenzione

Il nome predefinito dell'attributo è spesso parte dell'interfaccia. Se i dati usano `cost` ma la funzione legge `weight`, bisogna passare il parametro appropriato o rinominare l'attributo. Un attributo ignorato può produrre un risultato formalmente valido ma riferito a un problema diverso.

7 Separare algoritmo e libreria

Per ciascun argomento è utile affiancare:

1. una versione didattica, breve e tracciabile;
2. la funzione NetworkX per il confronto;
3. una funzione indipendente di verifica.

La versione didattica deve restituire non solo la soluzione finale, ma anche informazioni sulle iterazioni: nodi etichettati, archi rilassati, cammini aumentanti, alberi di base o costi ridotti. La visualizzazione non deve essere incorporata nel nucleo algoritmico: è meglio restituire dati strutturati e disegnarli in una funzione separata.

Schema di una funzione didattica

Una funzione per Dijkstra può restituire `dist`, `pred` e una lista `trace`. Ogni elemento di `trace` contiene il nodo reso permanente e una copia delle etichette dopo i rilassamenti. Lo stesso schema permette di costruire tabelle o animazioni senza modificare l'algoritmo.

8 Visualizzazione delle reti

Una figura efficace deve distinguere struttura e soluzione:

- posizione dei nodi salvata e riutilizzata;
- etichette degli archi coerenti, per esempio costo/capacità o flusso/capacità;
- cammino, albero o taglio evidenziato con colore e spessore;
- archi non rilevanti attenuati, senza eliminare informazione necessaria.

Listing 5: Evidenziare un cammino

```
def draw_path(G, path, weight="weight"):
    pos = nx.spring_layout(G, seed=7)
    path_edges = set(zip(path, path[1:]))
    colors = ["#820024" if e in path_edges else "#bbbbbb"
              for e in G.edges]
    widths = [2.8 if e in path_edges else 1.0 for e in G.edges]

    nx.draw(G, pos, with_labels=True, node_color="#f9f1f3",
            edge_color=colors, width=widths, arrows=True)
    labels = nx.get_edge_attributes(G, weight)
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
    plt.tight_layout()
```

Per un taglio minimo si colorano diversamente i nodi di W e $\overline{W}$; per un flusso si usa l'etichetta x_{ij}/u_{ij} ; per il CPM si evidenziano gli archi a slittamento nullo.

9 Test automatici

I test su istanze piccole proteggono da errori di segno e regressioni. Sono particolarmente utili proprietà che devono valere per ogni soluzione.

Listing 6: Esempi di test con pytest

```
def test_maxflow_equals_mincut(G):
    value, _ = nx.maximum_flow(G, "s", "t")
    cut_value, _ = nx.minimum_cut(G, "s", "t")
    assert value == cut_value

def test_shortest_path_cost(G):
    path = nx.shortest_path(G, "s", "t", weight="weight")
    cost = nx.path_weight(G, path, weight="weight")
    assert cost == nx.shortest_path_length(
        G, "s", "t", weight="weight"
    )

def test_tree_size(T):
    assert nx.is_tree(T)
    assert T.number_of_edges() == T.number_of_nodes() - 1
```

Con dati in virgola mobile si usa una tolleranza, per esempio `pytest.approx` o `math.isclose`. È utile includere anche casi non validi: domanda sbilanciata, capacità negativa, DAG con ciclo e Dijkstra con peso negativo.

10 Struttura della repository GitHub

Una possibile organizzazione è:

```
ottimizzazione-su-reti/
|-- README.md
|-- pyproject.toml
|-- src/network_optimization/
|   |-- validation.py
|   |-- visualization.py
|   |-- flows.py
|   |-- shortest_paths.py
```

```
|  '-- cpm.py
|-- notebooks/
|  |-- 01_grafi.ipynb
|  |-- 02_flusso_costo_minimo.ipynb
|  |-- 03_massimo_flusso.ipynb
|  |-- 04_cammini_minimi.ipynb
|  '-- 05_cpm.ipynb
|-- data/
|-- tests/
'-- exercises/
```

Il README dovrebbe indicare installazione, convenzioni, ordine dei notebook e licenza. I notebook illustrano gli esempi; le funzioni riutilizzabili appartengono a `src`; i test non vanno nascosti nei notebook.

11 Riproducibilità e stile

- fissare il seme dei layout casuali;
- non dipendere dall'ordine accidentale di nodi o archi;
- evitare variabili globali e celle eseguite fuori ordine;
- aggiungere docstring, type hints e nomi coerenti;
- salvare dati di esempio in formati testuali semplici;
- confrontare implementazione didattica e libreria senza assumere che restituiscano lo stesso cammino in caso di ottimi multipli.

12 Errori frequenti

1. Cambiare segno alle domande prima di passarle a NetworkX.
2. Usare `Graph` quando il modello richiede archi orientati.
3. Confondere `weight` e `capacity`.
4. Fidarsi del risultato senza verificare bilanci e vincoli.
5. Confrontare cammini o alberi invece dei valori quando esistono ottimi multipli.
6. Incorporare disegno, input e algoritmo nella stessa funzione.
7. Usare Dijkstra senza controllare i pesi.
8. Costruire il CPM su un grafo ciclico.
9. Non fissare il layout, ottenendo figure diverse a ogni esecuzione.

13 Esercizi di laboratorio

Esercizio 9.1 – Importazione. Definire un formato CSV per nodi e archi, importare una rete e validarne gli attributi.

Esercizio 9.2 – Flusso. Risolvere un flusso a costo minimo con `network_simplex` e verificarne indipendentemente bilanci e costo.

Esercizio 9.3 – Certificato. Calcolare massimo flusso e taglio minimo, visualizzando la partizione e gli archi del taglio.

Esercizio 9.4 – Confronto. Implementare Dijkstra didattico con traccia e confrontare valori e predecessori con NetworkX.

Esercizio 9.5 – CPM. Produrre una tabella pandas con *ES*, *EF*, *LS*, *LF* e slittamento di ogni attività.

Esercizio 9.6 – Test. Scrivere almeno un test positivo e uno negativo per ciascun algoritmo.

14 Sintesi del capitolo

Obiettivi di apprendimento

- Una convenzione comune rende interoperabili modelli, esempi e notebook.
- NetworkX risolve il grafo fornito, non verifica la correttezza del modello.
- Ogni soluzione dovrebbe essere accompagnata da controlli e certificati.
- Algoritmo, validazione e visualizzazione vanno mantenuti separati.
- I test automatici rendono affidabili le implementazioni didattiche.
- La repository deve distinguere codice riutilizzabile, notebook, dati ed esercizi.

Passo successivo. La struttura proposta costituisce la base naturale per realizzare la pagina GitHub del corso e pubblicare progressivamente notebook, istanze ed esercizi.